

HR & PAYROLL MANAGEMENT PLATFORM

High-Level Design (HLD)
Sprint 1 — Minimum Viable Product

Target Organization

Software solutions and software development company delivering customer-desired web, mobile, enterprise, integration, and custom software.

HLD Item	MVP Assumption
Industry	Software Solutions / Software Development
Workforce	Developers, QA, UI/UX, Project Managers, DevOps, HR, Finance, Sales, Support
Payroll	Monthly; MMK by default
Timezone	Asia/Yangon by default
Work Model	Office, Hybrid, Remote-ready
Architecture	Responsive Web + REST API + Modular Monolith
Primary Database	PostgreSQL

Changelog — v1.1

Adds real-time GPS capture on attendance check-in/check-out so employees can check in remotely from any location ("check in from everywhere"). Location is captured for HR visibility and audit only — it never blocks a check-in in this release. See updated Section 3 (scope), Section 11 (Attendance Architecture — new §11.3/§11.4), Section 17 (Security), Section 26 (ADR-009), Appendix A, and Appendix D.

Table of Contents

1. Document Purpose and Design Goals
2. Business and Product Context
3. MVP Scope and System Boundary
4. Architecture Overview
5. Logical Architecture
6. Deployment Architecture
7. Domain / Module Architecture
8. Core Data Architecture
9. Entity Relationship Overview
10. Authentication and Authorization Architecture
11. Attendance Architecture v1.1
12. Leave and Overtime Workflow Architecture
13. Payroll Architecture
14. API Architecture
15. Frontend Architecture
16. Transaction and Consistency Strategy
17. Security Architecture v1.1
18. Audit and Observability
19. Error Handling and Validation
20. Non-Functional Requirements
21. Scalability and Extension Strategy
22. Backup and Recovery
23. Testing Architecture v1.1
24. CI/CD and Environment Strategy
25. Sprint 1 Implementation Roadmap
26. Architecture Decision Records v1.1
27. Definition of Done
- Appendix A — Core Data Entities v1.1
- Appendix B — API Surface
- Appendix C — Key State Machines
- Appendix D — Future Architecture v1.1

1. Document Purpose and Design Goals

This HLD defines the technical structure for the Sprint 1 MVP of a web-based HR & Payroll Management Platform. It is intended as a practical reference for database design, backend services, APIs, frontend screens, testing, deployment, and future extension work.

The design uses a modular monolith rather than microservices. This keeps the MVP operationally simple while preserving domain boundaries for future extraction if scale or integration needs justify it.

Primary design goals

- Payroll, leave, attendance, and employee-data correctness.
- Clear separation between authentication, HR master data, transactions, and payroll.
- Strong organization/tenant isolation.
- Server-side business rules; frontend is never the source of truth.
- Historical payroll remains stable after later salary changes.
- Easy extension into recruitment, performance, expenses, assets, analytics, and AI.

2. Business and Product Context

The organization is a software solutions company that develops customer-requested digital products. Typical teams include software engineering, QA, UI/UX, project management, DevOps, HR, Finance, Sales, and Support. Employees may work from the office, remotely, or under a hybrid arrangement.

Business Area	MVP Representation
Software Delivery	Employees are organized by department and position; work model is recorded in employment configuration.
People Operations	Employee master data, attendance, leave, overtime, and approvals.
Finance/Payroll	Salary profiles, monthly payroll runs, deductions, allowances, and payslips.
Management	Operational dashboard and controlled approval actions.
Employee Self-Service	Attendance, requests, leave balance, and own payslip.

MMK is the default payroll currency, Asia/Yangon is the default organization timezone, and payroll is monthly. These are configuration values, not hard-coded rules.

3. MVP Scope and System Boundary

In scope

Sprint 1 proves the following end-to-end workflow:

```
HR creates employee
↓
Employee account activated
↓
Check-in / Check-out (optionally with GPS location)
↓
Leave / Overtime request
↓
HR approval
↓
Salary + approved OT + deductions
↓
Monthly payroll calculation
↓
Payroll approval
↓
Employee payslip
```

- Authentication & RBAC
- Organization settings
- Departments & positions
- Employee management
- Attendance, including optional real-time GPS capture on check-in/check-out v1.1
- Leave management
- Overtime
- Salary profiles
- Monthly payroll
- Payslips
- HR and employee dashboards
- Audit logs
- Validation, testing and deployment

Out of scope for Sprint 1

- Recruitment/ATS
- KPI/performance appraisal
- Learning management
- Asset management
- Expense claims
- Biometric devices

- Face recognition
- Hard geofence enforcement (blocking check-in by location) — **v1.1** log-only GPS capture is in scope; see §11.3/§11.4
- Bank disbursement integrations
- AI HR agent
- Advanced statutory payroll automation

v1.1 scope change: GPS/geofencing was listed as out of scope in Version 1.0. Real-time GPS capture on check-in/check-out is now in scope as a *log-only, non-blocking* feature so employees can check in from any location. Enforcing a geofence radius that rejects a check-in remains explicitly out of scope and is deferred to Appendix D.

4. Architecture Overview

High-level architecture

```
+-----+
| Web Browser / Client                               |
| React + TypeScript + Tailwind                     |
+-----+
| HTTPS / JSON                                       |
v
+-----+
| Node.js API Application                           |
| Express + TS                                       |
| Auth | Organization | Employees | Attendance | Leave |
| OT | Salary | Payroll | Payslip | Dashboard      |
+-----+
|
| +-----+
| v               v
+-----+ +-----+
| PostgreSQL | | Optional Infrastructure|
| transactional data| | Redis / Object Storage|
+-----+ +-----+
```

The initial deployment can run frontend/static assets and the API behind a reverse proxy. PostgreSQL is the system of record. Redis and object storage should be introduced only when a concrete requirement exists.

5. Logical Architecture

Use four conceptual layers inside the backend:

Layer	Responsibility	Should Not Do
Controller / Route	HTTP parsing, auth context, response mapping	Contain payroll business rules
Service / Domain	Business rules, orchestration, state transitions	Know HTTP-specific details
Repository / Data Access	Queries, persistence, transaction participation	Decide authorization policy
Infrastructure	DB, cache, email, storage, external adapters	Contain HR domain decisions

Recommended backend module structure

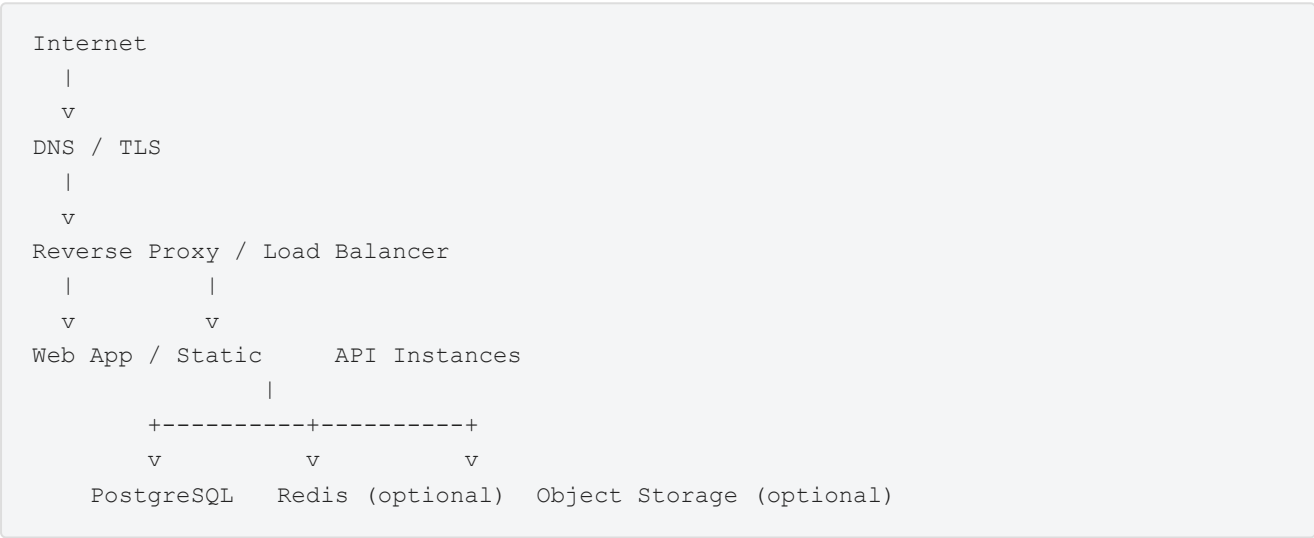
```
src/  
  modules/  
    auth/  
    organizations/  
    departments/  
    positions/  
    employees/  
    attendance/  
    leave/  
    overtime/  
    salary/  
    payroll/  
    payslips/  
    dashboard/  
    audit/  
  common/  
    middleware/  
    errors/  
    validation/  
  auth/  
  utils/  
  database/  
  config/  
  app.ts  
  server.ts
```

6. Deployment Architecture

Recommended environments

Environment	Purpose
Local	Developer machine; Docker Compose for PostgreSQL and optional Redis.
Development	Shared integration environment for active development.
Staging	Production-like release candidate testing.
Production	Company HR operations.

Production topology



Sprint 1 may use a single API instance, but the API should remain stateless so multiple instances can be introduced later.

7. Domain / Module Architecture

Module	Owns	Depends On
Auth	Identity, sessions/tokens, credentials	Users
Organization	Company settings, timezone, currency, location capture policy	Auth
Department/Position	Organizational structure	Organization
Employee	Employment profile, lifecycle	Organization, User
Attendance	Daily work sessions, optional GPS location metadata	Employee, organization settings
Leave	Types, balances, requests, approvals	Employee, policy
Overtime	OT requests and approval	Employee, salary policy
Salary	Compensation profiles/effective dates	Employee, organization
Payroll	Runs/items/calculation snapshots	Employee, salary, leave, OT
Payslip	Released salary statements	Payroll
Dashboard	Read models/aggregates	Relevant domains
Audit	Sensitive action trace	All sensitive modules

8. Core Data Architecture

Use PostgreSQL for the transactional core. Relational integrity is valuable because payroll and leave require consistency across related records.

Principle	Design
Primary keys	UUID recommended for external-facing identifiers.
Money	DECIMAL/NUMERIC; never floating point.
Time	Store timestamps consistently; display in organization timezone.
Dates	Use DATE for leave days and payroll periods.
Deletion	Prefer status/deactivation for master data with history.
Constraints	Use unique and foreign-key constraints for invariants.
Indexes	Index organization_id and common filtering columns.

Common fields where appropriate

```
id
organization_id
created_at
updated_at
created_by
updated_by
status / is_active
```

9. Entity Relationship Overview

Conceptual ERD

```
Organization
|--- Department
|--- Position
|--- User -----
|
|--- Employee <-----
|
|--- AttendanceRecord (+ optional GPS lat/lng/accuracy on check-in and check-out)
|--- LeaveBalance --- LeaveType
|--- LeaveRequest --- LeaveType
|--- OvertimeRequest
|--- SalaryProfile

Organization
|--- PayrollRun
|--- PayrollItem --- Employee
|--- Payslip

All sensitive operations ---> AuditLog
```

A PayrollItem stores a snapshot of calculation inputs. Historical payroll must not depend on today's mutable SalaryProfile.

10. Authentication and Authorization Architecture

Role	Organization	Employees	Attendance	Leave/OT	Salary	Payroll	Audit
Super Admin	Full	Full	Full	Full	Full	Full	Full
HR Admin	Manage	Full	Manage	Approve	Manage	Manage	Read
Employee	Own	Own only	Own	Own requests	Own limited view	Own payslips	Relevant only

Authorization flow

```
HTTP Request
↓
Authenticate identity
↓
Resolve organization scope
↓
Check role / permission
↓
Validate resource ownership / scope
↓
Execute service operation
↓
Audit if sensitive
↓
Return response
```

Authentication identifies the caller; authorization decides what the caller may do. Both are enforced on the server.

11. Attendance Architecture

Attendance sequence

```
Employee → POST /attendance/check-in { lat?, lng?, accuracyMeters? }
↓
Validate active employee
↓
Check no open session
↓
Create AttendanceRecord (store location metadata if provided and capture enabled)
↓
Return current state

Employee → POST /attendance/check-out { lat?, lng?, accuracyMeters? }
↓
Validate open session
↓
Calculate duration
↓
Store check-out location metadata if provided
↓
Close record
```

Rule	Implementation
Duplicate check-in	Guard against more than one open session.
Check-out	Requires matching open session.
Timezone	Business date uses organization timezone.
Correction	HR-only operation with mandatory audit reason.
Status	Derive from schedule/leave policy where possible.

11.3 Remote GPS Check-In v1.1

Goal: an employee can check in and check out from anywhere, while HR gains visibility into where a check-in happened. The design intentionally keeps location as an *informational signal*, not an access-control gate.

- The client may attach GPS coordinates (`lat`, `lng`, `accuracyMeters`) to the check-in and check-out calls, captured via the browser Geolocation API at the moment the employee taps the action.
- Coordinates are optional on every request. A missing, denied, or failed GPS read never blocks or delays a check-in — the record is simply stored with `locationSource = UNAVAILABLE`.
- Organization-level setting `locationCaptureEnabled` (default `true`) lets an organization turn location capture off entirely for privacy reasons; when off, the server discards any coordinates the client sends and stores nothing.
- Location is never used to reject, delay, or flag a check-in as invalid in this release. There is no distance/radius check in the request path.

11.4 Location Data Handling and Future Geofencing v1.1

Concern	MVP Design
Storage	checkInLat/Lng/AccuracyMeters and checkOutLat/Lng/AccuracyMeters on AttendanceRecord; see Appendix A.
Read access	HR Admin, Super Admin, and the owning employee only. Excluded from bulk employee directory exports by default.
Accuracy	Stored as reported by the device; no minimum-accuracy rejection. The UI may surface a "low accuracy" hint above a configurable threshold.
Trust model	Client-supplied coordinates are not cryptographically verifiable. Treat them as an operational/HR visibility signal, not a security control.
Hard geofencing	Deliberately not built in this release. Appendix D documents it as a future optional layer (reject or flag check-ins outside a configured radius for office-model employees only).

Design rationale: a strict geofence would contradict the stated goal ("check in from everywhere") for hybrid/remote employees and would require handling GPS spoofing, indoor accuracy loss, and false rejections before it could be trusted to block anything. Log-only capture delivers the visibility value immediately without those risks; enforcement can be layered in later behind an explicit organization opt-in.

12. Leave and Overtime Workflow Architecture

Leave state machine

```
PENDING --approve--> APPROVED
|
|--reject--> REJECTED |--cancel--> CANCELLED
|
|--cancel--> CANCELLED
```

Balance consumption and request status transition must occur in one transaction.

Overtime state machine

```
PENDING --approve--> APPROVED --> PAYROLL INPUT
|
|--reject--> REJECTED
|--cancel--> CANCELLED
```

Concern	HLD Rule
Overlap	Reject overlapping requests according to policy.
Approval	Only authorized roles can approve/reject.
Concurrency	Use transactions/locking/versioning to prevent double consumption.
Payroll eligibility	Only APPROVED OT and approved leave affect payroll.

13. Payroll Architecture

Payroll is the most sensitive domain in the MVP. Treat calculation as a deterministic domain service and persist its result as a snapshot.

Payroll flow

```
Create Payroll Run
↓
Validate period / employee eligibility
↓
Load SalaryProfile effective for period
↓
Load approved OT
↓
Load approved leave
↓
Calculate earnings
↓
Calculate deductions
↓
Validate results
↓
Persist PayrollItems
↓
CALCULATED → HR APPROVE → APPROVED → PAID
```

Calculation model

```
Gross = Basic Salary + Fixed Allowances + Approved Overtime + Bonus
Net = Gross - Fixed Deductions - Unpaid Leave Deduction - Other Authorized Deductions
```

State	Allowed Operations
DRAFT	Build/update inputs; calculate.
CALCULATED	Review; recalculate before approval.
APPROVED	Read/release; no ordinary recalculation.
PAID	Historical read-only; future adjustment/reversal required.

14. API Architecture

Use versioned REST endpoints.

Domain	Representative Endpoint
Auth	POST /api/v1/auth/login; GET /api/v1/auth/me
Employees	GET/POST /api/v1/employees; PATCH /api/v1/employees/:id
Attendance	POST /api/v1/attendance/check-in; POST /api/v1/attendance/check-out (both accept optional lat/lng/accuracyMeters in the body)
Leave	POST /api/v1/leave/requests; POST /api/v1/leave/requests/:id/approve
Overtime	POST /api/v1/overtime/requests; POST /api/v1/overtime/requests/:id/approve
Salary	GET/PATCH /api/v1/employees/:id/salary-profile
Payroll	POST /api/v1/payroll/runs; POST /api/v1/payroll/runs/:id/calculate; POST /api/v1/payroll/runs/:id/approve
Payslip	GET /api/v1/payslips; GET /api/v1/payslips/:id
Dashboard	GET /api/v1/dashboard/hr; GET /api/v1/dashboard/me

Example response envelope

```
{
  "success": true,
  "data": {...},
  "meta": {...}
}

{
  "success": false,
  "error": {
    "code": "LEAVE_BALANCE_INSUFFICIENT",
    "message": "Insufficient leave balance"
  }
}
```

15. Frontend Architecture

Recommended frontend structure

```
src/  
  app/  
    routes/  
    layouts/  
  features/  
    auth/  
    employees/  
    attendance/  
    leave/  
    overtime/  
    payroll/  
    payslips/  
    dashboard/  
  components/  
  hooks/  
  lib/  
    api/  
    schemas/  
    types/
```

Concern	Recommendation
Routing	Protected route boundaries.
Server state	TanStack Query.
Forms	React Hook Form + Zod.
UI state	Local state or focused store; do not duplicate server state globally.
Authorization	Hide unavailable UI but enforce permissions on backend.
Tables	Server-side pagination/filtering for HR lists.
Money	Format API values; do not calculate business results in presentation components.
Geolocation v1.1	Request the browser Geolocation API only at the moment of the check-in/check-out action (not on page load), with a short timeout; silently fall back to no location on denial, timeout, or error; never block the action button on a pending permission prompt.

16. Transaction and Consistency Strategy

Use database transactions when multiple records must change together.

Operation	Transaction Boundary
Approve leave	Request + balance consumption + audit.
Cancel approved leave	Request + balance restoration + audit.
Calculate payroll	Payroll items + calculation snapshot.
Approve payroll	State validation + update + audit.
Mark paid	State validation + update + audit.

Concurrency controls

- Use database constraints for structural invariants.
- Use row locks or optimistic versioning for balances and payroll state transitions.
- Make approval operations state-aware to prevent double approval.
- Never rely on a previous GET to prove a write is still valid.

17. Security Architecture

Risk	Control
Credential theft	Strong password hashing, secure reset tokens, rate limiting.
Salary exposure	RBAC + ownership + organization scope.
Tenant leakage	organization_id enforced at service/repository boundary.
Injection	Parameterized queries/ORM + validated input.
Secrets	Environment/secret manager; never source control.
Sensitive logs	Redact credentials, tokens, bank data and unnecessary payroll values.
Historical manipulation	Payroll snapshots and controlled state transitions.
Location data exposure v1.1	Read access limited to HR Admin/Super Admin and the owning employee; excluded from bulk directory exports; organization may disable capture entirely (locationCaptureEnabled=false); coordinates are informational only and never gate access, attendance validity, or payroll.

An employee's permission to view their own profile does not imply permission to view another employee's compensation — the same principle applies to another employee's raw GPS coordinates.

18. Audit and Observability

Audit at minimum: employee creation/update/deactivation; salary changes; leave/OT approvals; attendance corrections; payroll calculation/approval/payment changes; role changes; sensitive organization setting changes (including changes to `locationCaptureEnabled`).

Operational observability should include structured logs, request correlation IDs, error tracking, health/readiness endpoints, database metrics, and basic API latency/error metrics.

19. Error Handling and Validation

HTTP	Use
400	Malformed or invalid request
401	Authentication required/invalid
403	Authenticated but forbidden
404	Resource not found in authorized scope
409	Conflict/current-state or uniqueness violation
422	Business rule cannot be satisfied
500	Unexpected server failure

Validation should exist at four levels: frontend UX validation, API schema validation, domain/business validation, and database constraints. Note: GPS coordinates are validated for shape only (numeric range) — an out-of-range or missing value never produces a 422 in this release, since location cannot invalidate a check-in.

20. Non-Functional Requirements

Category	Sprint 1 Target
Performance	Typical CRUD API under ~500 ms in normal load, excluding heavy reports.
Scalability	Stateless API; indexed relational DB; ready for horizontal scaling.
Security	RBAC, tenant isolation, secure credentials, audit trail.
Maintainability	Domain modules, typed contracts, tests, documented APIs.
Recoverability	Automated backups and tested restore before production use.
Usability	Responsive web UI for desktop and common mobile browser sizes.
Localization	MMK and Asia/Yangon defaults; no hard-coded locale assumptions.

21. Scalability and Extension Strategy

- Add Redis when caching, queues, rate limiting, or distributed coordination is justified.
- Introduce background jobs for email, notifications, PDF generation, and heavy reports.
- Use object storage for employee documents and generated files.
- Use read models/materialized views for analytics.
- Extract a domain into a service only when scale, cadence, or integration boundaries justify it.
- Keep domain services independent from Express.

Future domains

Future	Integration Point
Recruitment/ATS	Employee onboarding
Performance/KPI	Employee and organization
Expenses	Payroll/Finance
Assets	Employee
Notifications	Cross-domain event layer
AI HR Agent	Authorized domain tools
Bank disbursement	Payroll payment adapter
Geofence enforcement v1.1	Attendance module, org-level opt-in

22. Backup and Recovery

- PostgreSQL is the authoritative transactional source.
- Use automated production backups.
- Define retention based on business/legal requirements.
- Test restoration, not only backup creation.
- Define RPO/RTO before production launch.
- Do not store the only copy of payroll documents on an application container.

23. Testing Architecture

Test Layer	Examples
Unit	Payroll formulas, leave-day calculation, state transitions.
Service	Approve leave, calculate payroll, employee lifecycle.
Repository	Organization scoping, constraints, query correctness.
API integration	Authentication, RBAC, validation.
E2E	Onboarding → attendance → leave/OT → payroll → payslip.
Security	Cross-organization, cross-employee, privilege escalation tests.

Critical invariants

- One open attendance session per employee/day.
- Leave approval cannot consume balance twice.
- Cancelled approved leave restores balance exactly once.
- One payroll run per organization and period under the selected payroll type.
- Approved/paid payroll cannot be changed by ordinary calculation endpoints.
- Historical payroll remains unchanged after salary profile changes.
- Employee cannot access another employee's payslip.
- **v1.1** Check-in/check-out succeeds identically whether or not GPS coordinates are supplied.
- **v1.1** Employee cannot read another employee's raw GPS coordinates.
- **v1.1** When `locationCaptureEnabled=false`, no coordinates are persisted even if the client sends them.

24. CI/CD and Environment Strategy

Recommended pipeline

```
Pull Request
↓
Lint → Typecheck → Unit Tests
↓
Build → Integration Tests
↓
Container Build
↓
Development
↓
Staging / Release Candidate
↓
Production Approval
↓
Deploy + Migration + Health Check
```

Database migrations must be version-controlled and executed explicitly during deployment. Do not make undocumented production schema edits.

25. Sprint 1 Implementation Roadmap

Order	Work Package	Deliverable
1	Foundation	Repository, Docker, TypeScript, linting, CI, config
2	Database	PostgreSQL schema, migrations, seed data
3	Auth/RBAC	Login, session/token, role guards
4	Organization	Settings, departments, positions
5	Employee	CRUD, lifecycle, account linking
6	Attendance	Check-in/out (incl. optional GPS capture), history, HR correction
7	Leave	Types, balances, requests, approval
8	Overtime	Requests, approval, payable hours
9	Salary	Profiles, effective dates
10	Payroll	Run lifecycle, calculation, snapshots
11	Payslip	Employee release/view
12	Dashboard	HR/employee operational views
13	Hardening	Audit, security, tests
14	Release	Staging, migration, deployment, demo

Build vertically where possible: complete a thin end-to-end slice rather than building all frontend screens first. This exposes integration issues early.

26. Architecture Decision Records

ADR	Decision	Reason
ADR-001	Modular monolith	Fast MVP development with clear domain boundaries.
ADR-002	PostgreSQL	Relational integrity and transactions for payroll/HR.
ADR-003	REST API	Simple browser integration and clear contracts.
ADR-004	TypeScript end-to-end	Strong typing and validation discipline.
ADR-005	Payroll snapshots	Protect historical financial records.
ADR-006	Organization-scoped data	Future multi-tenant/product evolution.
ADR-007	Server-side business rules	Prevent client manipulation and duplication.
ADR-008	Stateless API	Future horizontal scaling.
ADR-009 v1.1	Log-only GPS capture; no geofence enforcement in MVP	Support genuine remote check-in ("check in from everywhere") without hard-blocking legitimate work; avoid trusting unverifiable client-supplied coordinates as a security gate; keep hard geofencing an explicit, optional future extension rather than default behavior.

27. Definition of Done

- All Sprint 1 modules have authenticated and authorized APIs.
- Database migrations reproduce the schema from a clean environment.
- Organization scoping is enforced server-side.
- Critical rules have automated tests.
- Payroll uses decimal arithmetic and immutable calculation snapshots.
- Approval transitions are protected against concurrent actions.
- Sensitive operations are audited.
- Frontend supports primary HR and employee workflows responsively.
- API errors are consistent and documented.
- CI passes lint, tests, and build.
- Staging deployment succeeds using production-like migrations.
- End-to-end demo completes employee creation → attendance → leave/OT → payroll → payslip.
- **v1.1** Remote GPS check-in captures and stores location metadata (when permitted and enabled) without blocking or delaying any legitimate check-in, including from remote/hybrid employees.
- **v1.1** Raw GPS coordinates are visible only to HR Admin, Super Admin, and the owning employee.

Appendix A — Core Data Entities

Entity	Key Fields / Notes
Organization	id, name, timezone, currency, payroll_cycle, locationCaptureEnabled v1.1 , status
User	id, organization_id, email, password_hash, role, status
Department	id, organization_id, name, status
Position	id, organization_id, department_id, title, status
Employee	id, organization_id, user_id, employee_no, join_date, department_id, position_id, work_model, status
AttendanceRecord	id, organization_id, employee_id, work_date, check_in, check_out, working_minutes, check_in_lat, check_in_lng, check_in_accuracy_m, check_out_lat, check_out_lng, check_out_accuracy_m, location_source (GPS MANUAL UNAVAILABLE) v1.1
LeaveType	id, organization_id, name, paid, policy settings
LeaveBalance	id, employee_id, leave_type_id, period, entitled, used, remaining
LeaveRequest	id, employee_id, leave_type_id, start_date, end_date, days, status, approved_by
OvertimeRequest	id, employee_id, date/start/end, hours, rate/multiplier, status
SalaryProfile	id, employee_id, effective_from/to, basic_salary, allowances, deductions, OT settings
PayrollRun	id, organization_id, period, status, totals, approved_by, paid_at
PayrollItem	id, payroll_run_id, employee_id, snapshot inputs, earnings, deductions, gross, net
Payslip	id, payroll_item_id, released_at, document reference if generated
AuditLog	id, organization_id, actor_id, action, resource_type, resource_id, metadata, timestamp

Appendix B — API Surface

Domain	Method	Path
Auth	POST	/api/v1/auth/login
Auth	GET	/api/v1/auth/me
Employees	GET/POST	/api/v1/employees
Employees	GET/PATCH	/api/v1/employees/:id
Attendance	POST	/api/v1/attendance/check-in (body: lat?, lng?, accuracyMeters?)
Attendance	POST	/api/v1/attendance/check-out (body: lat?, lng?, accuracyMeters?)
Leave	POST	/api/v1/leave/requests
Leave	POST	/api/v1/leave/requests/:id/approve
OT	POST	/api/v1/overtime/requests
OT	POST	/api/v1/overtime/requests/:id/approve
Salary	GET/PATCH	/api/v1/employees/:id/salary-profile
Payroll	POST	/api/v1/payroll/runs
Payroll	POST	/api/v1/payroll/runs/:id/calculate
Payroll	POST	/api/v1/payroll/runs/:id/approve
Payroll	POST	/api/v1/payroll/runs/:id/mark-paid
Payslip	GET	/api/v1/payslips
Payslip	GET	/api/v1/payslips/:id
Dashboard	GET	/api/v1/dashboard/hr
Dashboard	GET	/api/v1/dashboard/me
Organization v1.1	GET/PATCH	/api/v1/organization (includes locationCaptureEnabled)

Appendix C — Key State Machines

Employee

```
DRAFT → ACTIVE → INACTIVE
      |
      TERMINATED
```

Leave Request

```
PENDING → APPROVED → CANCELLED
      |
      |--> REJECTED
      |--> CANCELLED
```

Payroll Run

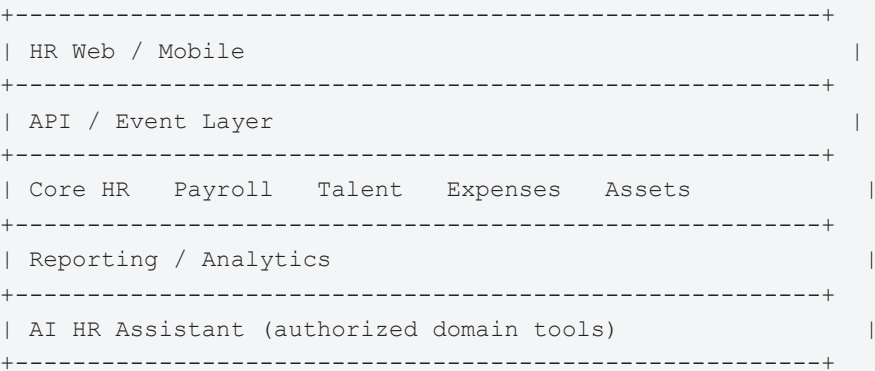
```
DRAFT → CALCULATED → APPROVED → PAID
```

Attendance

```
NO_SESSION → OPEN → CLOSED
```

Location fields (lat/lng/accuracy/source) are attributes captured at OPEN and CLOSED transitions; they do not add new states.

Appendix D — Future Architecture



The AI layer should never bypass authorization or write directly to payroll tables. It should call permission-aware application services/tools. This preserves the same business rules whether an action starts from a UI, API client, scheduled job, or AI assistant.

v1.1 future extension — optional geofence enforcement:

- Add an org-level geofenceZones table (label, lat, lng, radiusMeters) and a locationPolicy enum (NONE | LOG_ONLY | GEOFENCE_ENFORCED) on Organization.
- When GEOFENCE_ENFORCED, apply the check only to employees with work_model = OFFICE; remote and hybrid employees remain exempt by design.
- A rejected check-in would use the existing 422 business-rule error channel with a new code (e.g. OUTSIDE_GEOFENCE), consistent with how leave/payroll already surface business-rule failures.
- This must ship behind an explicit organization opt-in — it must never become default behavior, per ADR-009.